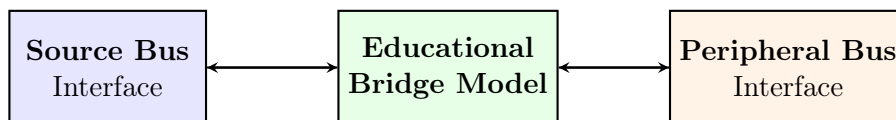


Educational Bus Bridge Model for Verification (AHB-like to APB-like Example)

Version 1.1

Author: Vikash Kumar
Date: April 2026
Status: Released
Document ID: EDU-BRIDGE-MODEL-001



This document describes an independently developed educational bridge model used to demonstrate verification methodology concepts. It is not an official Arm[®], AMBA[®], AHB, or APB specification and is not intended to represent a production implementation. Familiar industry signal names are used only to support learning, verification framework evaluation, and methodology illustration.

See the **Disclaimer and Intellectual Property Notice** page that follows for the full notice.

Disclaimer and Intellectual Property Notice

Copyright

Copyright © 2026 Vikash Kumar. All rights reserved.

No part of this document may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

Disclaimer

The views expressed in this document are those of the author and do not represent the position of any employer, standards body, or organization. No endorsement, sponsorship, or affiliation with any employer or organization is implied. This work was developed by the author in personal capacity and outside any employer-funded project. References to industry protocols (such as AHB and APB) are based on publicly available information only and are intended for educational purposes.

Third-Party Intellectual Property Notice

This document may reference third-party technologies, standards, trademarks, or intellectual property, including but not limited to Arm[®] AMBA[®] protocols (e.g., AHB, APB). All such rights remain the property of their respective owners. No license, express or implied, is granted by this document to any third-party intellectual property, including but not limited to that of Arm Limited or its affiliates. Use of any third-party technology or specification referenced herein may require a separate license from the applicable rights holder.

Revision History

Version	Date	Author	Description
1.0	Feb 2026	Vikash Kumar	Initial release
1.1	Apr 2026	Vikash Kumar	Added formal Disclaimer and Third-Party Intellectual Property Notice

Contents

Disclaimer and Intellectual Property Notice	1
1 Introduction	5
1.1 Purpose	5
1.2 Important Notice	5
1.3 Scope	5
1.4 Definitions	5
1.5 References	5
2 Functional Overview	6
2.1 Conceptual Block Diagram	6
2.2 Model Summary	6
2.3 Intended Use	6
3 Interface View	7
3.1 Source-side Interface	7
3.2 Peripheral-side Interface	7
3.3 Clock and Reset Notes	8
4 Example Register View	8
4.1 Register Philosophy	8
4.2 Register Map	8
4.3 CTRL Register	9
4.4 STATUS Register	9
4.5 Captured Debug Registers	9
5 Behavioral Model	9
5.1 Core Translation Behavior	9
5.2 Sequence Handling	10
5.3 Back-Pressure and Wait Behavior	10
5.4 Representative Transfer Timing	10
5.5 Representative Wait-State Timing	11
6 Reset and Initialization	11
6.1 Reset Intent	11
6.2 Reset State	11
6.3 Soft Reset	11
7 Error and Exception Scenarios	12
7.1 Modeled Error Classes	12
7.2 Error Handling Intent	12
7.3 Representative Error Timing	12
8 Verification Usage	12
8.1 Why This Model Exists	12
8.2 Typical Verification Artifacts Derived from the Model	13
8.3 Example Verification Questions	13
A Example Timing Vocabulary	13

B Example State Machine View	14
C Publication Note	14
D Revision History	14

1 Introduction

1.1 Purpose

This document presents a simplified bus bridge model used to exercise verification workflows. The model converts transactions from a high-performance source-side interface into accesses on a peripheral-side interface.

The primary goal of this document is educational. It provides a structured artifact that can be used to demonstrate:

- AI-assisted verification planning
- interface and transaction modeling
- assertion and coverage development
- debug and trace analysis workflows
- student and early-career engineer learning

1.2 Important Notice

This document is not a formal protocol standard and does not reproduce any official specification. It is an independently written educational model. References to widely used bus terminology are included only to help readers connect the example to familiar verification scenarios.

1.3 Scope

This model covers the following behavior at a conceptual level:

- a single source-side interface driving a single peripheral-side interface
- 32-bit address and data examples
- single transfers and simple multi-transfer sequences
- basic status and control register examples
- representative reset, wait, and error scenarios

This document does not attempt to define a complete or standards-accurate bus specification.

In particular, it does not attempt to exhaustively define:

- all legal source-side transaction encodings
- all corner-case burst behaviors
- arbitration or multi-target decoding
- clock-domain crossing
- production-grade security, parity, or ECC features

1.4 Definitions

Term	Meaning in this document
Source bus	A high-throughput request interface used as model input
Peripheral bus	A simpler target interface used as model output
Bridge model	Educational logic translating source traffic to target accesses
Sequence	A related set of transfers processed in order
DUT	Device under test
FSM	Finite state machine

1.5 References

1. Publicly available bus protocol literature and training materials

2. VEGA framework user material
3. Cognitive Verification Architecture educational content

2 Functional Overview

2.1 Conceptual Block Diagram

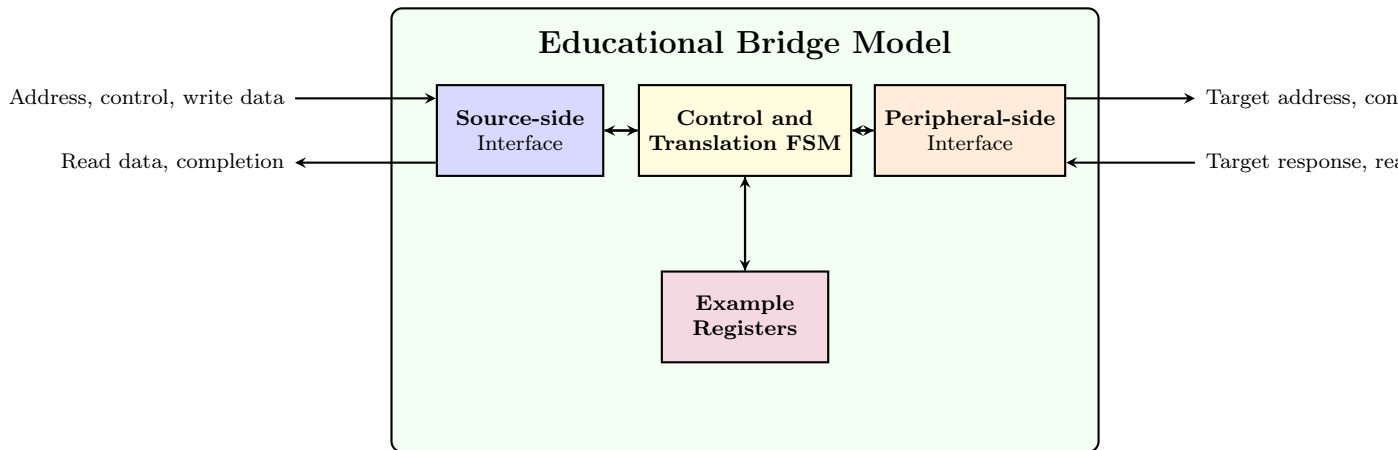


Figure 1: Conceptual educational bridge organization

2.2 Model Summary

Aspect	Educational intent
Protocol translation	Show how a bridge can map source-side requests to peripheral accesses
Address/data examples	Use 32-bit examples for readability
Sequence handling	Demonstrate ordered transfer handling in a simple bridge
Back-pressure	Model target-side stalls and source-side completion delay
Error behavior	Exercise timeout, invalid-address, and target-error scenarios
Register examples	Provide software-visible state for debug and observability

Table 1: Educational feature summary

2.3 Intended Use

This model is intended to support:

- verification test plan creation
- assertion development
- functional coverage definition
- AI-assisted trace interpretation
- classroom and workshop exercises

3 Interface View

3.1 Source-side Interface

The source-side interface uses familiar industry signal names to make the example easy to follow. In this document, these signals are treated as a learning-oriented abstraction rather than a formal protocol definition.

Signal	Width	Dir	Educational description
HCLK	1	In	Clock used for the example timing model
HRESETn	1	In	Active-low reset used for initialization scenarios
HADDR	32	In	Source-side address example
HTRANS	2	In	Indicates transfer phase intent, such as new or continued activity
HWRITE	1	In	Direction indicator for read or write examples
HSIZE	3	In	Example transfer-size control
HBURST	3	In	Example sequence-pattern control
HWDATA	32	In	Source-side write payload
HSEL	1	In	Local selection control for the bridge model
HREADY_IN	1	In	Upstream completion context input
HRDATA	32	Out	Source-side read payload returned by the model
HREADY_OUT	1	Out	Model completion indicator / back-pressure output
HRESP	1	Out	Simplified status indicator for success or error

Table 2: Source-side example signals

3.2 Peripheral-side Interface

The peripheral-side interface represents a simpler two-phase transfer style, again using familiar names only for educational readability.

Signal	Width	Dir	Educational description
PCLK	1	In	Target-side clock, commonly tied to HCLK in this example
PRESETn	1	In	Active-low reset for target-side model state
PADDR	32	Out	Target-side address example
PSEL	1	Out	Indicates active target selection in the example flow
PENABLE	1	Out	Marks the active transfer phase on the target side
PWRITE	1	Out	Target-side direction indicator
PWDATA	32	Out	Target-side write payload
PRDATA	32	In	Target-side read payload
PREADY	1	In	Target completion / stall indicator
PSLVERR	1	In	Simplified target-side error indicator

Table 3: Peripheral-side example signals

3.3 Clock and Reset Notes

- This educational model assumes a single clock domain for simplicity.
- Reset behavior is intentionally simplified to support debug and verification exercises.
- Readers should treat timing and reset behavior here as representative rather than normative.

4 Example Register View

4.1 Register Philosophy

The model includes a small set of example control and status registers to support observability and software-driven verification scenarios. These are not intended to mirror any commercial register map.

4.2 Register Map

Offset	Name	Access	Purpose in the model
0xF00	CTRL	R/W	Enable and control knobs
0xF04	STATUS	R/W1C	Status and sticky error flags
0xF08	ERROR_ADDR	RO	Captured address for debug
0xF0C	ERROR_INFO	RO	Captured context for debug

Table 4: Example register map

4.3 CTRL Register

Bits	Name	Access	Reset	Purpose
31:8	Reserved	RO	0	Reserved in this model
7	TIMEOUT_EN	R/W	0	Enables timeout behavior in verification runs
6:4	TIMEOUT_VAL	R/W	3'b111	Selects a timeout window for experiments
3	ERR_INT_EN	R/W	0	Enables sticky interrupt-style indication
2	Reserved	RO	0	Reserved
1	SOFT_RST	R/W	0	Triggers a model-local reset action
0	ENABLE	R/W	1	Enables bridge activity in the example flow

Table 5: Example CTRL register

4.4 STATUS Register

Bits	Name	Access	Reset	Purpose
31:8	Reserved	RO	0	Reserved in this model
7	ERR_INT	R/W1C	0	Sticky aggregated error indication
6	TIMEOUT_ERR	R/W1C	0	Timeout scenario observed
5	PSLVERR	R/W1C	0	Target-side error observed
4	ADDR_ERR	R/W1C	0	Invalid-address scenario observed
3:2	Reserved	RO	0	Reserved
1	BUSY	RO	0	Model currently processing traffic
0	READY	RO	1	Model available for new work

Table 6: Example STATUS register

4.5 Captured Debug Registers

Register	Purpose
ERROR_ADDR	Captures the address associated with the most recent modeled error event
ERROR_INFO	Captures context such as transfer direction, approximate beat index, and coarse error class

Table 7: Captured debug state

5 Behavioral Model

5.1 Core Translation Behavior

At a high level, the model accepts source-side transactions and turns them into target-side accesses. Each accepted source-side transfer is converted into one target-side operation. The bridge can delay source-side completion when the target side is not ready.

- FLOW-1:** Capture source-side address, control, and payload when the model accepts a transfer.
FLOW-2: Translate the transfer into a target-side setup phase.
FLOW-3: Advance to a target-side active phase.
FLOW-4: Hold progress if the target side indicates it is not ready.
FLOW-5: Complete the transaction and return any read data or error context.

5.2 Sequence Handling

The model supports simple ordered multi-transfer sequences. In these sequences:

- the model tracks whether traffic is part of a continued stream
- addresses may advance according to transfer size
- the model may emulate wrapped or bounded sequences for verification purposes

This section is intentionally conceptual. Exact sequence encoding and all legal corner cases are outside the scope of this educational artifact.

5.3 Back-Pressure and Wait Behavior

When the target side is not ready, the bridge model keeps the target transfer active and delays source-side completion. This enables verification engineers to exercise:

- stalled completion behavior
- timeouts
- scoreboard synchronization
- latency-aware debug

5.4 Representative Transfer Timing

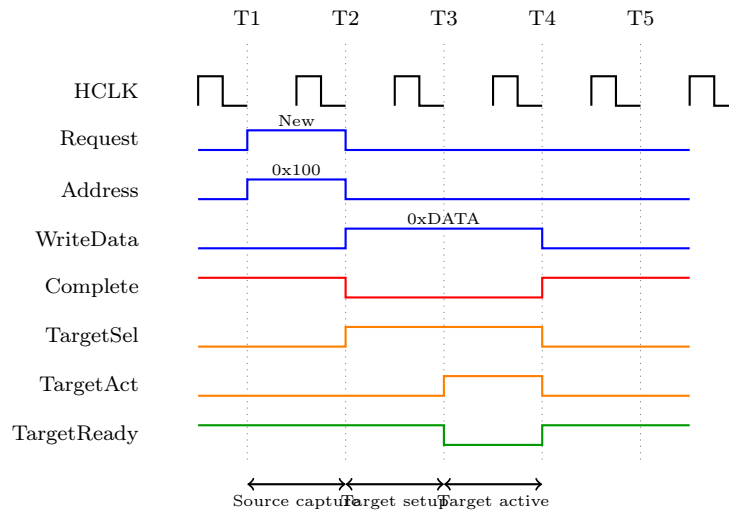


Figure 2: Representative transfer timing for the educational model

5.5 Representative Wait-State Timing

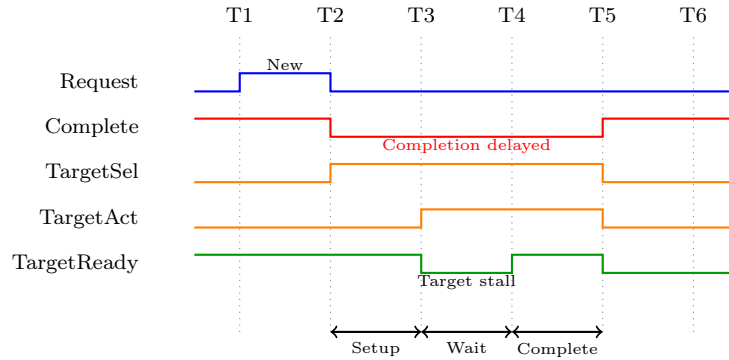


Figure 3: Representative delayed-completion timing

6 Reset and Initialization

6.1 Reset Intent

Reset is modeled to return the bridge to a clean idle state. The goal is to support verification scenarios such as:

- reset during idle
- reset during active transfer
- reset followed by immediate re-use
- reset interaction with sticky error state

6.2 Reset State

Signal/Register	Example Reset Value	Notes
HREADY_OUT	1	Ready for new activity
HRESP	0	No error indication
HRDATA	0	Cleared for simplicity
PSEL	0	No active target selection
PENABLE	0	No active target phase
PADDR	0	Cleared for simplicity
PWDATA	0	Cleared for simplicity
CTRL	0x00000001	Enabled, timeout disabled
STATUS	0x00000001	Ready, no sticky error set
ERROR_ADDR	0x00000000	Cleared
ERROR_INFO	0x00000000	Cleared

Table 8: Example reset state

6.3 Soft Reset

A model-local soft reset may be triggered through the example control register. This is useful for software-driven tests and debug reproducibility. The soft reset action is intended to:

- clear active transfer state
- clear sticky status flags

- preserve basic configuration where appropriate

7 Error and Exception Scenarios

7.1 Modeled Error Classes

The educational model includes a small number of coarse error classes:

Error class	Representative meaning in the model
Address error	Request targeted an unsupported or invalid range in the example map
Timeout error	Target-side readiness did not arrive within the programmed observation window
Target error	Peripheral-side error indication was observed during an active access

Table 9: Modeled error classes

7.2 Error Handling Intent

When an error is observed, the model:

- records debug context
- updates sticky status bits
- returns a simplified source-side error indication
- becomes ready for subsequent debug or re-test activity

7.3 Representative Error Timing

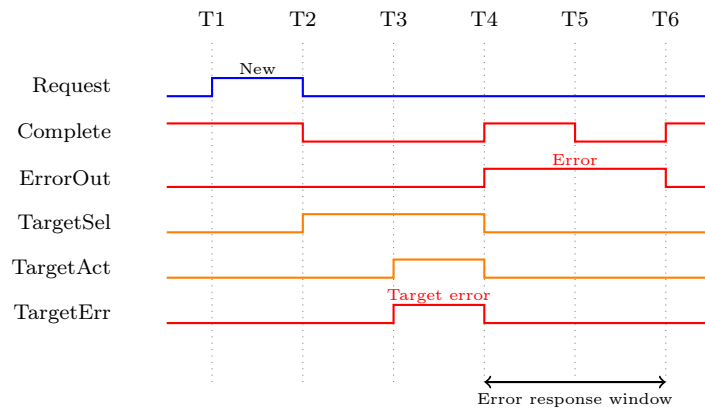


Figure 4: Representative modeled error timing

8 Verification Usage

8.1 Why This Model Exists

This model was created to support verification framework development rather than to act as a shipping design document. In particular, it is useful for:

- converting informal requirements into checkable behavior

- deriving assertions from interface intent
- generating coverage goals for bridge activity
- creating AI-assisted debug prompts from observed traces
- demonstrating how an educational specification can drive verification assets

8.2 Typical Verification Artifacts Derived from the Model

Artifact	Use in the VEGA / cognitive verification flow
Assertions	Check setup/active progression, completion gating, and error capture
Coverage items	Measure reset, wait-state, error, and sequence scenario exploration
Sequences	Generate single, delayed, and error-injected transactions
Debug prompts	Correlate model state with trace signatures and expected outcomes

Table 10: Representative verification outputs

8.3 Example Verification Questions

- Does source-side completion remain blocked while the target side stalls?
- Is read data returned only after target-side completion?
- Are sticky error flags captured and preserved until cleared?
- Does reset cleanly abort in-flight work?
- Are multi-transfer sequences handled in-order?

A Example Timing Vocabulary

The diagrams in this document use a simplified timing vocabulary:

- **Source capture:** source-side request is accepted by the model
- **Target setup:** target-side address and control are prepared
- **Target active:** target-side transfer is in progress
- **Completion delayed:** source-side completion is held off by target latency
- **Error response window:** source-side error signaling interval in the model

B Example State Machine View

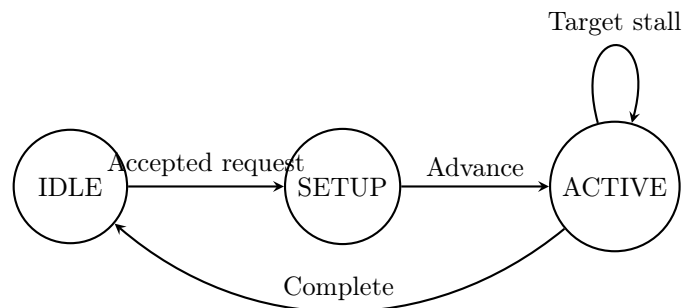


Figure 5: Simplified educational state machine

C Publication Note

This document is intentionally simplified to support teaching, verification planning, and AI-assisted methodology discussion. It should be read as an educational bridge model, not as a standards document or production design specification.

D Revision History

Version	Date	Author	Changes
1.0	Feb 2026	Vikash Kumar	Initial release
1.1	Apr 2026	Vikash Kumar	Added formal Disclaimer and Third-Party Intellectual Property Notice

Table 11: Document revision history